

**WYDZIAŁ FIZYKI
I INFORMATYKI STOSOWANEJ**

Porównanie metod pruningu na CPU i GPU

Jakub Ledwoń, Piotr Nowak

AGH University of Krakow

June 8, 2026

1 Porównanie metod pruningu na CPU i GPU

1.1 Cel eksperymentu

Celem przeprowadzonych badań było porównanie dwóch metod redukcji złożoności sieci neuronowych:

- **Unstructured Pruning** – usuwanie pojedynczych wag o najmniejszym znaczeniu,

- **Structured Pruning** – usuwanie całych kanałów i filtrów sieci.

Analizę przeprowadzono na dwóch platformach sprzętowych:

- procesorze centralnym (**CPU**),
- procesorze graficznym (**GPU**).

Wszystkie modele były trenowane przez 5 epok na zbiorze CIFAR-10. W przypadku pruning’u strukturalnego zastosowano dodatkowy etap fine-tuningu również trwający 5 epok. Dla każdego wariantu zmierzono:

- liczbę parametrów modelu,
- liczbę operacji MAC (Multiply-Accumulate),
- czas wykonania pojedynczego przejścia w przód,
- skuteczność klasyfikacji.

Dodatkowo dla metod pruningu wyznaczono charakterystyki ROC dla wszystkich klas.

2 Konfiguracja eksperymentu

Wszystkie eksperymenty przeprowadzono z wykorzystaniem konwolucyjnej sieci neuronowej (CNN) przeznaczonej do klasyfikacji obrazów ze zbioru CIFAR-10. Model został zaimplementowany w architekturze składającej się z czterech warstw konwolucyjnych oraz dwóch warstw w pełni połączonych (feed-forward).

We wszystkich warstwach ukrytych zastosowano funkcję aktywacji ReLU. Ostatnia warstwa sieci generuje predykcje dla 10 klas odpowiadających kategoriom występującym w zbiorze CIFAR-10.

Architektura modelu przedstawia się następująco:

1. Warstwa konwolucyjna operująca na obrazach wejściowych o rozmiarze 32×32 .
2. Warstwa konwolucyjna z operacją max-pooling, redukująca rozmiar map cech do 16×16 .
3. Warstwa konwolucyjna z operacją max-pooling, redukująca rozmiar map cech do 8×8 .
4. Warstwa konwolucyjna z operacją max-pooling, redukująca rozmiar map cech do 4×4 .
5. Warstwa w pełni połączona przekształcająca tensor o wymiarze $256 \times 4 \times 4$ do wektora o długości 512.
6. Warstwa wyjściowa w pełni połączona realizująca odwzorowanie $512 \rightarrow 10$.

W przypadku pruningu strukturalnego redukowano liczbę kanałów w warstwach konwolucyjnych, zachowując ogólną strukturę architektury. Dzięki temu możliwe było zmniejszenie liczby parametrów oraz liczby operacji MAC przy jednoczesnym utrzymaniu funkcjonalności modelu.

3 Wyniki eksperymentów

3.1 Model bazowy (bez pruningu)

Tabela 1 przedstawia parametry modelu referencyjnego.

Table 1: Parametry modelu bazowego

Liczba parametrów	3.25 M
Liczba MAC	155.90 M

Table 2: Porównanie czasu wykonania modelu bazowego

Platforma	Czas [s]
GPU	0.000598
CPU	0.046818

Można zauważyć, że GPU wykonuje inferencję około 78 razy szybciej od CPU.

3.2 Unstructured Pruning

W przypadku pruningu niestrukturalnego wyzerowano około 50% wag modelu.

Table 3: Statystyki sparsity dla Unstructured Pruning

Liczba wszystkich parametrów	3 249 994
Liczba wyzerowanych parametrów	1 623 392
Sparsity	49.95%

Pomimo wyzerowania połowy wag, liczba parametrów oraz liczba operacji MAC pozostają niezmienione:

- Parametry: 3.25 M,
- MAC: 155.90 M.

Wynika to z faktu, że architektura modelu nie została fizycznie zmodyfikowana. Wagi zostały jedynie wyzerowane.

Jednak uzyskana sparsity na poziomie bliskim 50% pokazuje, że połowa parametrów nie wnosi już informacji do modelu. Taka reprezentacja może zostać wykorzystana w dalszych etapach optymalizacji, np. przez kompresję modelu lub zastosowanie wyspecjalizowanych bibliotek obsługujących macierze rzadkie.

Table 4: Porównanie czasu wykonania po Unstructured Pruning

Platforma	Czas [s]
GPU	0.000370
CPU	0.047728

Na GPU zaobserwowano niewielkie przyspieszenie inferencji. Na CPU czas wykonania pozostał praktycznie niezmieniony.

3.3 Structured Pruning

W przypadku pruningu strukturalnego usunięto 20% kanałów konwolucyjnych oraz odpowiednio zmniejszono warstwy klasyfikatora.

Liczba kanałów została zmniejszona według schematu:

$$64 \rightarrow 48, \quad 128 \rightarrow 96, \quad 256 \rightarrow 200$$

Dzięki temu rzeczywista architektura modelu została uproszczona.

Table 5: Wpływ Structured Pruning na złożoność modelu

Parametr	Przed	Po
MAC	155.90 M	90.60 M
Liczba parametrów	3.25 M	1.99 M

Structured pruning zmniejszył:

- liczbę parametrów o około 38.8%,
- liczbę operacji MAC o około 41.9%.

Table 6: Porównanie czasu wykonania po Structured Pruning

Platforma	Czas [s]
GPU	0.000742
CPU	0.015260

Na CPU zaobserwowano znaczące przyspieszenie działania modelu. Czas inferencji został skrócony ponad trzykrotnie.

Na GPU nie uzyskano analogicznego przyspieszenia. Wynika to najprawdopodobniej z niewielkiego rozmiaru modelu oraz narzutu związanego z uruchamianiem operacji CUDA.

4 Porównanie jakości klasyfikacji

Table 7: Porównanie dokładności klasyfikacji

Wariant	GPU	CPU
Bez pruningu	78%	79%
Unstructured Pruning	77%	77%
Structured Pruning	84%	86%

Można zauważyć, że:

- model bazowy osiąga wysoką skuteczność,
- unstructured pruning nie powoduje zauważalnych zmian jakości klasyfikacji,
- structured pruning prowadzi do znaczącej poprawy skuteczności modelu.

W przypadku structured pruning usunięcie kanałów sieci okazało się działać jak forma regularyzacji, eliminując nadmiarowe filtry i zmniejszając redundancję reprezentacji.

5 Analiza wpływu sparsity oraz rodzaju regularyzacji

Table 8: Dokładność klasyfikacji

	st_L1_20	st_L1_50	st_L2_50	st_L1_80	st_L2_80	unst_20	unst_50	unst_80
GPU	0.83	0.84	0.83	0.73	0.73	0.78	0.78	0.67
CPU	0.83	0.83	0.83	0.73	0.73	0.77	0.74	0.65

Table 9: Czas inferencji dla pojedynczego batcha

	st_L1_20	st_L1_50	st_L2_50	st_L1_80	st_L2_80	unst_20	unst_50	unst_80
GPU	0.000788	0.000241	0.020574	0.000547	0.000523	0.000444	0.000731	0.000656
CPU	0.012720	0.020574	0.020574	0.004582	0.004824	0.167879	0.047936	0.047259

Table 10: Liczba operacji MAC

	st_L1_20	st_L1_50	st_L2_50	st_L1_80	st_L2_80	unst_20	unst_50	unst_80
GPU	90.60	39.68	39.68	4.80	4.80	155.90	155.90	155.90
CPU	90.60	39.68	39.68	4.80	4.80	155.90	155.90	155.90

Table 11: Liczba parametrów modelu

	st_L1_20	st_L1_50	st_L2_50	st_L1_80	st_L2_80	unst_20	unst_50	unst_80
GPU	1.99	0.82	0.82	0.11	0.11	3.25	3.25	3.25
CPU	1.99	0.82	0.82	0.11	0.11	3.25	3.25	3.25

W tej sekcji analizie poddano wyłącznie wpływ poziomu sparsity (20%, 50%, 80%) oraz rodzaju regularyzacji (L1 vs L2) w przypadku pruning’u strukturalnego.

Wyniki pokazują, że wzrost sparsity prowadzi do systematycznego spadku złożoności modelu, jednak wpływ na jakość zależy od stopnia redukcji. Dla poziomu 20% (najłagodniejszego pruning’u strukturalnego) uzyskiwana dokładność pozostaje porównywalna z modelem bazowym, a w niektórych przypadkach nawet nieznacznie wzrasta. Oznacza to, że niewielka redukcja kanałów działa jak łagodna forma regularyzacji, eliminując część nadmiarowych reprezentacji.

Przy sparsity 50% obserwuje się stabilność wyników jakościowych — dokładność pozostaje na podobnym poziomie dla wariantów L1 i L2. Sugeruje to, że w tym zakresie redukcji model nadal zachowuje wystarczającą pojemność reprezentacyjną, a różnice między regularyzacją L1 i L2 nie mają istotnego wpływu na końcową skuteczność klasyfikacji.

Dla sparsity 80% widoczny jest już wyraźny spadek jakości modelu. W tym przypadku redukcja struktury staje się na tyle agresywna, że zaczyna ograniczać zdolność modelu do reprezentowania bardziej złożonych cech. Zarówno L1, jak i L2 prowadzą do pogorszenia wyników, przy czym różnice między nimi pozostają niewielkie i nie wskazują jednoznacznej przewagi żadnej z metod regularyzacji.

Porównując L1 i L2 w całym zakresie sparsity, można zauważyć, że L1 (promująca większą rzadkość i bardziej agresywne zerowanie/wybieranie filtrów) nie daje istotnie innych wyników jakościowych niż L2 (bardziej równomierna penalizacja). W praktyce oznacza to, że w analizowanym modelu dominującym czynnikiem wpływającym na jakość jest poziom sparsity, a nie wybór rodzaju regularyzacji.

Podsumowując, kluczowym czynnikiem determinującym kompromis między złożonością a jakością jest stopień redukcji (20–50–80%), natomiast różnice między L1 i L2 mają drugorzędne znaczenie i ujawniają się jedynie w marginalnym stopniu.

6 Krzywe ROC

6.1 Model bazowy GPU

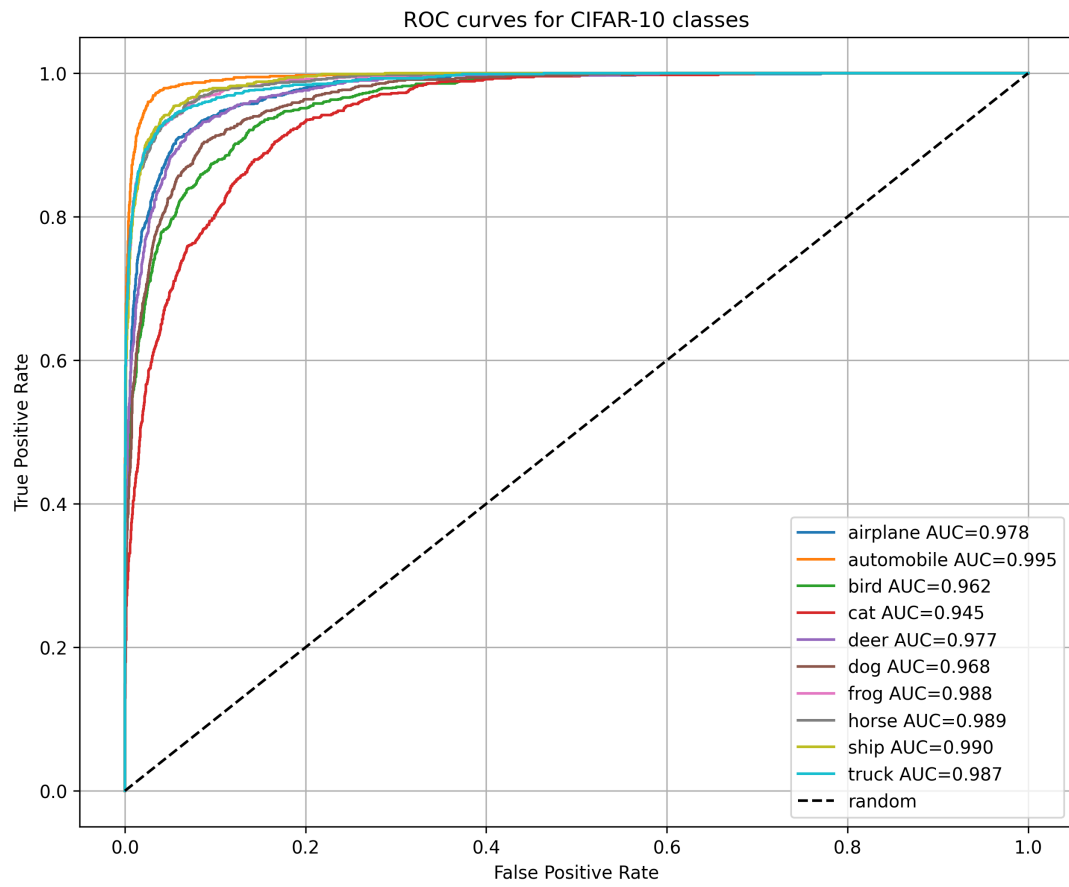


Figure 1: Krzywe ROC dla modelu bazowego uruchamianego na GPU

6.2 Unstructured Pruning GPU

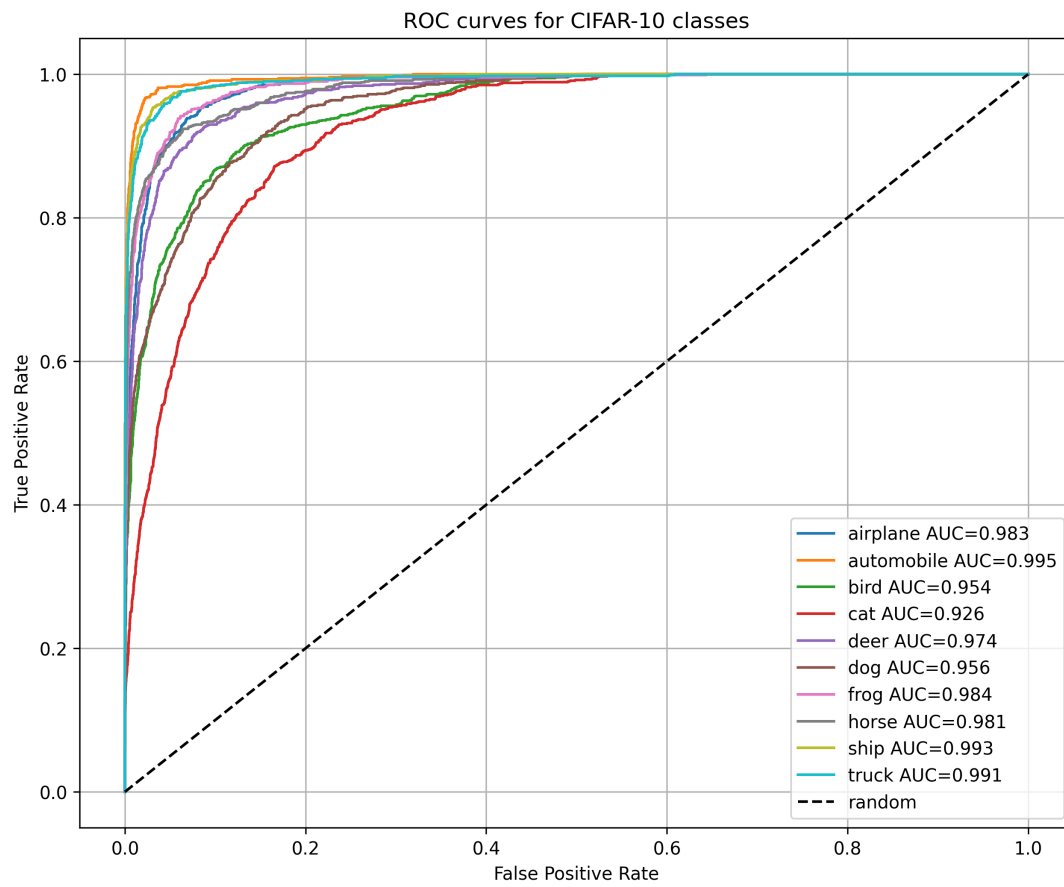


Figure 2: Krzywe ROC dla modelu po Unstructured Pruning na GPU

6.3 Structured Pruning GPU

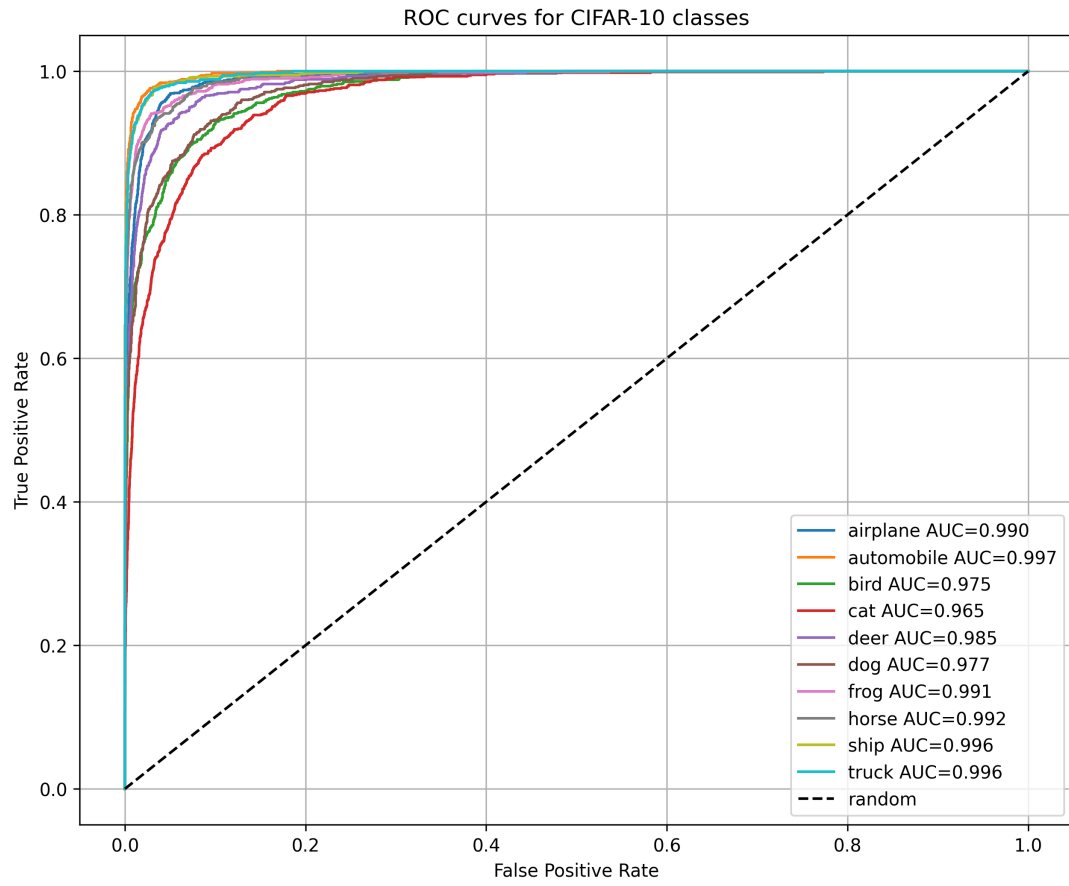


Figure 3: Krzywe ROC dla modelu po Structured Pruning na GPU

6.4 Model bazowy CPU

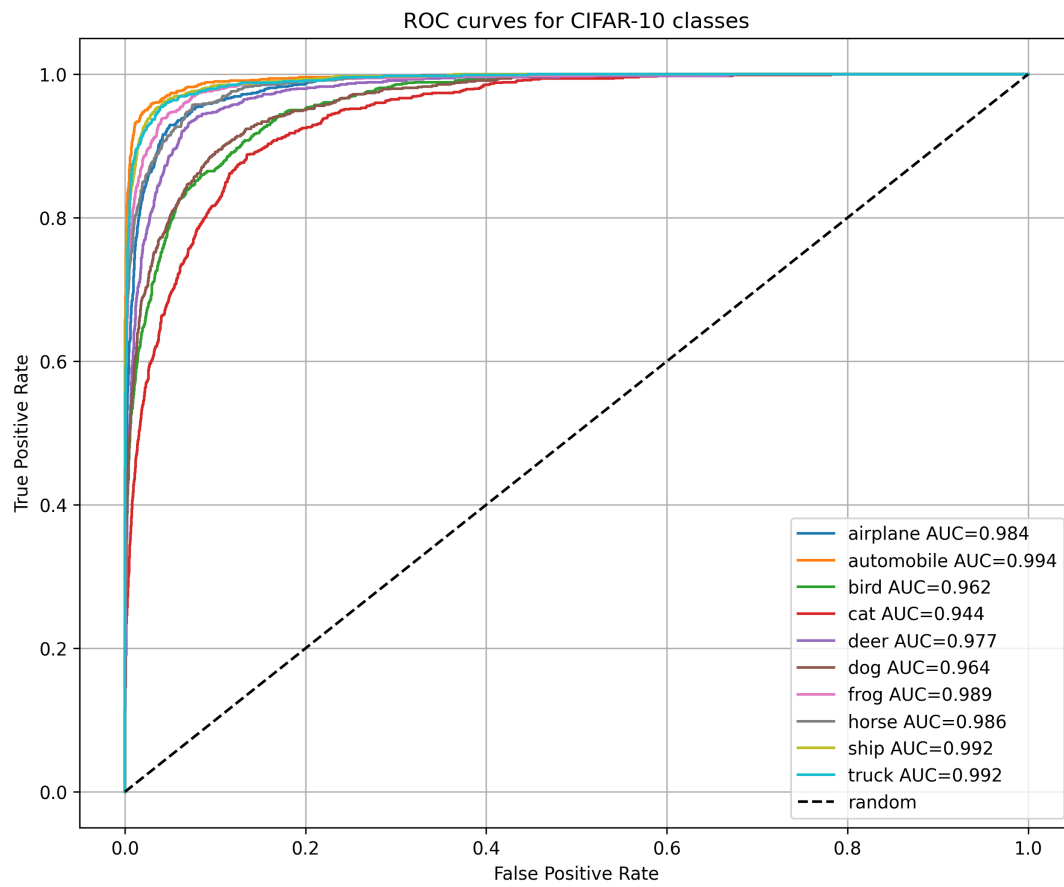


Figure 4: Krzywe ROC dla modelu bazowego uruchamianego na CPU

6.5 Unstructured Pruning CPU

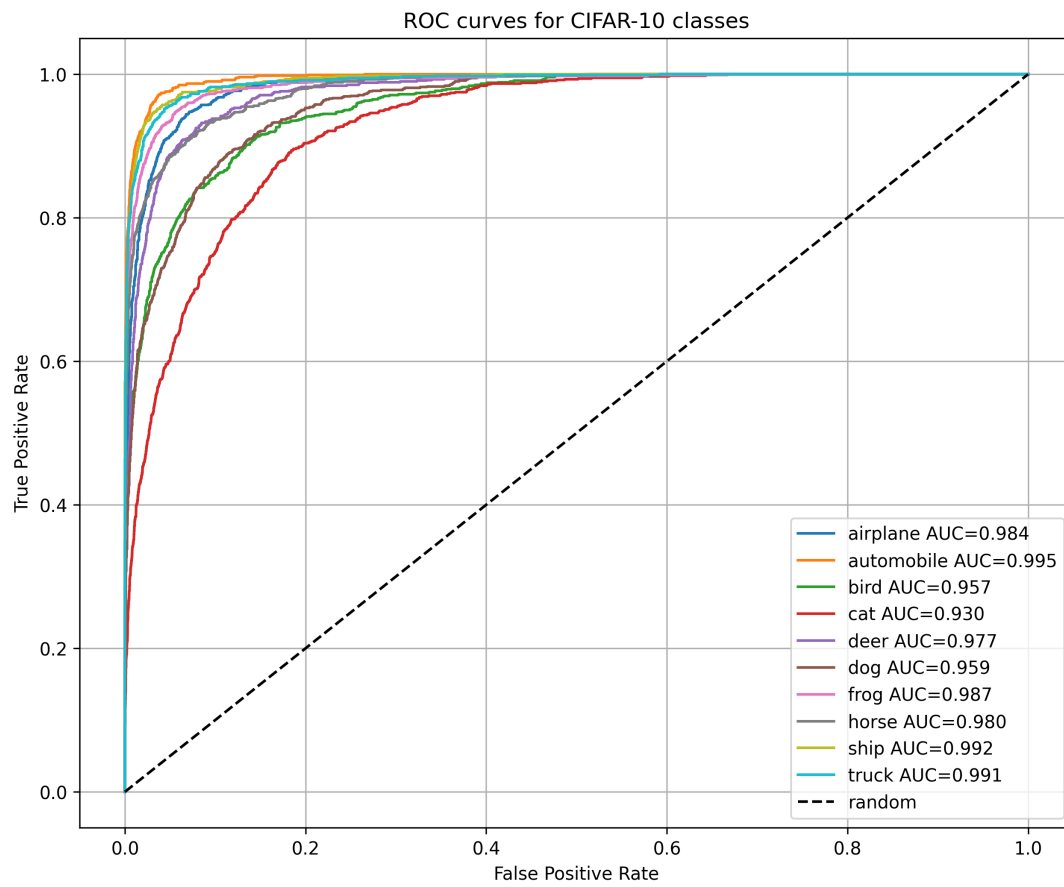


Figure 5: Krzywe ROC dla modelu po Unstructured Pruning na CPU

6.6 Structured Pruning CPU

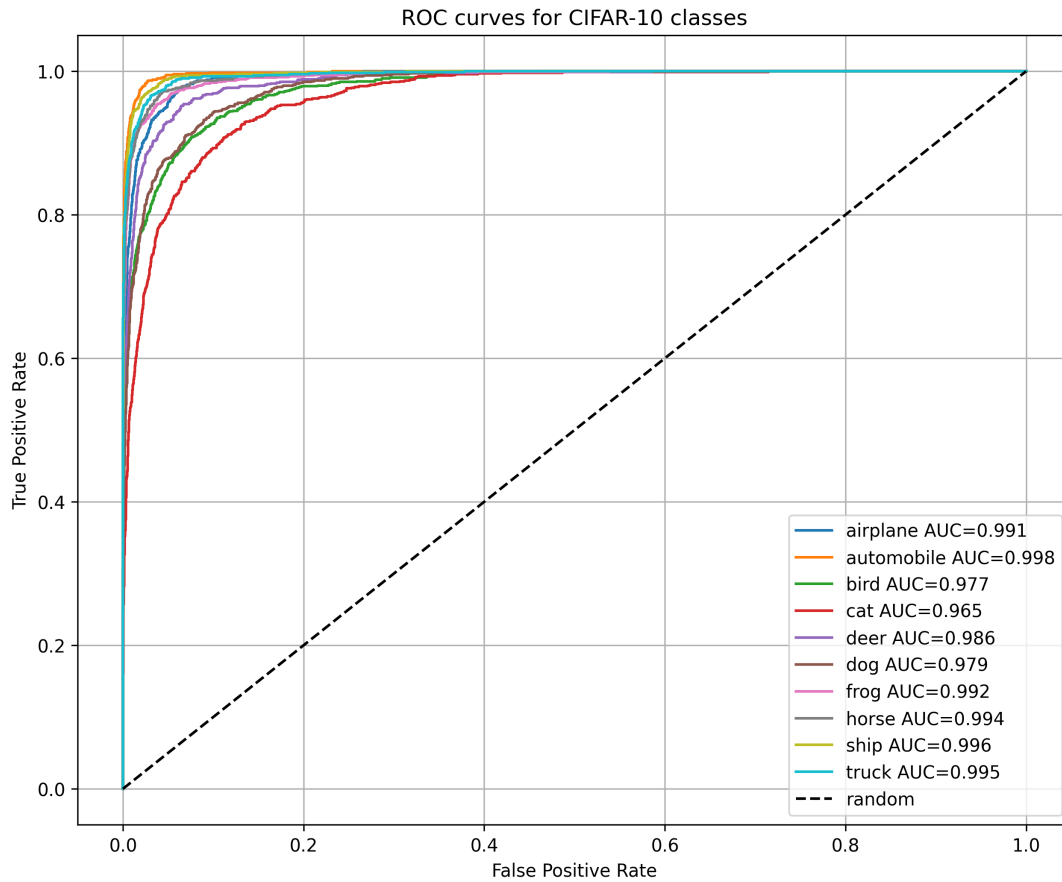


Figure 6: Krzywe ROC dla modelu po Structured Pruning na CPU

Krzywe ROC dla wszystkich wariantów są do siebie zbliżone. Żaden pruning nie spowodował pogorszenia zdolności rozróżniania klas, co jest zgodne z wynikami accuracy przedstawionymi wcześniej.

7 Wnioski

Przeprowadzone eksperymenty pokazują istotne różnice pomiędzy pruningiem strukturalnym i niestrukturnym.

Unstructured pruning pozwolił wyzerować około 50% parametrów modelu bez zmiany jego architektury. Dzięki temu uzyskano wysoką sparsity, jednak liczba parametrów oraz liczba operacji MAC pozostały niezmienione. Spadek skuteczności klasyfikacji był umiarkowany, szczególnie w przypadku uruchamiania modelu na CPU.

Structured pruning doprowadził do rzeczywistego uproszczenia architektury sieci. Liczba parametrów została zmniejszona z 3.25 M do 1.99 M, co oznacza redukcję o około 38.8%. Liczba operacji MAC spadła z 155.90 M do 90.60 M, czyli o około 41.9%.

Przełożyło się to na wyraźne przyspieszenie działania modelu, szczególnie na CPU, gdzie redukcja złożoności obliczeniowej bezpośrednio wpływa na czas inferencji. Na GPU różnice były mniejsze ze względu na narzut środowiska CUDA oraz równoległość obliczeń.

Co istotne, structured pruning nie spowodował pogorszenia jakości klasyfikacji. Zaobserwowano nawet poprawę dokładności względem modelu bazowego, co może wynikać z działania regularizacyjnego redukcji nadmiarowych kanałów i zmniejszenia przeuczenia.

Uzyskane wyniki pokazują więc, że w badanym eksperymencie pruning strukturalny okazał się bardziej efektywną metodą zarówno pod względem redukcji złożoności modelu, jak i jakości klasyfikacji.

Pruning niestukturalny może być natomiast korzystniejszy w sytuacjach, gdy dostępne są wyspecjalizowane implementacje obliczeń rzadkich (sparse kernels), pozwalające faktycznie wykorzystać zerowane wagi w obliczeniach, lub gdy kluczowym celem jest maksymalna kompresja modelu pod względem pamięciowym bez konieczności zmiany architektury. Jest on również często stosowany jako metoda stopniowej redukcji wag w procesie fine-tuningu dużych modeli, gdzie istotna jest minimalna ingerencja w strukturę sieci.